

MI2D: Accelerating Matrix Inversion with 2-Dimensional Tile Manipulations

Lingfeng Chen

Xi'an Jiaotong University, China
dfc201888@stu.xjtu.edu.cn

Wenzhe Zhao

Xi'an Jiaotong University, China
wenzhe@xjtu.edu.cn

Tian Xia

Xi'an Jiaotong University, China
tian_xia@xjtu.edu.cn

Pengju Ren

Xi'an Jiaotong University, China
pengjuren@xjtu.edu.cn

ABSTRACT

Matrix inversion is critical in mathematics and scientific applications. Large-scale dense matrix inversion is especially challenging for modern computers due to its heavy dependency of matrix elements and the poor temporal data locality. In this paper, we propose a novel accelerator termed MI2D, which converts matrix inversion into regular matrix multiplications using 2-dimensional cross-tile operations and novel algorithms for efficient data reuse and computations. Our evaluations show that MI2D can be easily integrated with existing matrix engines in modern high-end CPU and NPU, and effectively improves matrix inversion with 2.7× speedup against Intel Skylake CPU, and 24× against NVIDIA RTX 2080 Ti.

CCS CONCEPTS

• **Computer systems organization** → **Parallel architectures**;
• **Networks** → **Network architectures**; • **Hardware** → **Very large scale integration design**.

KEYWORDS

matrix inversion, architecture, on-chip network, accelerator

ACM Reference Format:

Lingfeng Chen, Tian Xia, Wenzhe Zhao, and Pengju Ren. 2022. MI2D: Accelerating Matrix Inversion with 2-Dimensional Tile Manipulations. In *Proceedings of the Great Lakes Symposium on VLSI 2022 (GLSVLSI '22)*, June 6–8, 2022, Irvine, CA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3526241.3530314>

1 INTRODUCTION

Matrix inversion plays important roles in scientific and engineering applications including the system optimization, electronic circuits, signal processing, etc. Matrix inversion is the bottleneck in many of these problems as it is usually called multiple times during the computation. Table 1 lists several applications where explicit matrix inversion is considered as the performance bottleneck. Many modern linear algebra libraries, including the Linear Algebra Package

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
GLSVLSI '22, June 6–8, 2022, Irvine, CA, USA.

© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9322-5/22/06...\$15.00
<https://doi.org/10.1145/3526241.3530314>

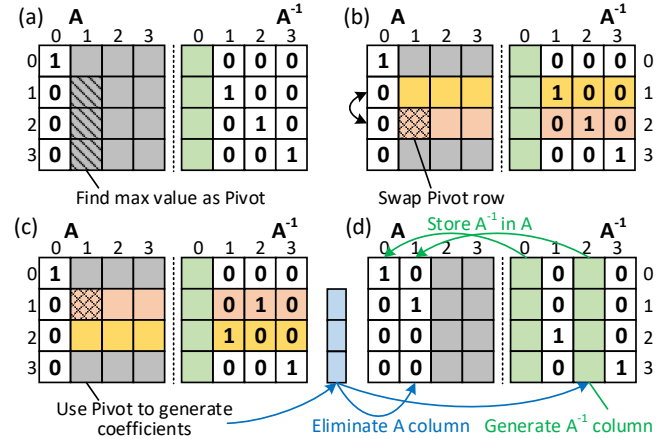


Figure 1: Process of matrix inversion using Gauss-Jordan Elimination with partial pivoting technique.

Table 1: Applications and problem sizes where matrix inversion is the bottleneck.

Applications	Matrix Scale Range
Massive MIMO Pre-coding[19][16]	40 – 400
Medical Image Recovery[14]	128 – 1024
ISS LQOC Control[3][1]	10^3
OFDM channel estimation[11]	512 – 8192
Electrical Circuit Simulation[3][1]	$10^4 - 10^6$

(LAPACK), Intel Math Kernel Library (MKL) and cuBLAS, provide matrix inversion functions based on Gauss-Jordan Elimination (GJE) as the following equations [5]:

$$P[A|I_A] = [PA|PI_A] = [I|A^{-1}] \quad (1)$$

$A \in \mathcal{R}^{N \times N}$ is the matrix to invert. $[A|I_A]$ is the augmented matrix with an identity matrix I_A attached to the right side of A . P is the permutation matrix of elementary row transformations that converts A into I , while converting I_A into A^{-1} . Normally the partial pivoting technique is also applied to suppress the error propagation during computation [2]. The process of GJE matrix inversion using partial pivoting is depicted in Fig 1, which includes four steps:

GJE-1 (Partial pivoting): when eliminating column k , instead of using $a_{k,k}$, search and find the largest absolute value in the partial column vector $A_{(k:N-1),k}$ as the pivot element.

GJE-2 (Pivot row swap): exchange the row that contains the selected pivot element with row k . After swap, the new $a_{k,k}$ element

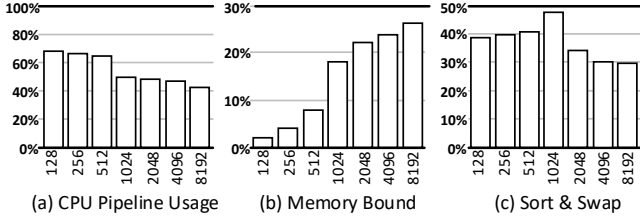


Figure 2: Performance breakdown of CPU pipeline usage, memory access bound and sorting/swap child functions on Intel MKL.

is the selected pivot element.

GJE-3 (Get coefficients): generate coefficients $c_{(0:N-1)}$ to eliminate other elements in the pivot's column (column k):

$$\forall a_{i,k} \in A_{(0:N-1),k}, c_i = \frac{a_{i,k}}{a_{i,k}} \quad (2)$$

GJE-4 (Elimination): to eliminate column k , process the elements in the pivot row $A_{k,(0:N-1)}$ as:

$$\forall a_{k,j} \in A_{k,(0:N-1)}, a_{k,j} = \frac{a_{k,j}}{a_{k,k}}. \quad (3)$$

And each other element undergoes:

$$\forall a_{i,j} \notin A_{k,(0:N-1)}, a_{i,j} = c_i a_{i,j} - a_{k,j}. \quad (4)$$

Step (1)-(4) repeat for N iterations. At each iteration, one column of A is eliminated, and one column of A^{-1} is generated. This new column can be saved in the space of the eliminated A 's column to reduce the memory footprint, as shown in Fig 1(d). As rows are randomly exchanged, the generation sequence of A^{-1} 's columns is out-of-order. Therefore, many approaches include an additional step to reorder A^{-1} 's columns to the correct position.

Modern computers usually use parallel processing units (e.g. GPU) or SIMD instructions (e.g. Intel AVX, ARM SVE) to accelerate the process. However, such methods tend to result in undermined efficiency. One critical problem is the low temporal locality as most elements are only used once for each iteration and may cause serious memory bandwidth bottleneck when the matrix is large. Another challenge is the random data manipulation caused by partial pivoting, which involves searching in random data length and random row exchange. For A^{-1} , the permutation of columns is also random. These random data operations further breaks the data locality and deteriorates the memory wall. Fig 2 reports the performance of Intel MKL matrix inversion kernel on Intel Skylake CPU on random FP32 matrices ranging from 128×128 to 8192×8192 . Metrics are collected using VTuneProfiler [15]. As shown in Fig 2(a)(b), when the matrix scale grows, CPU pipeline usage drops and memory bound gets more serious. For matrices larger than 1024, more than 50% of pipeline slots are wasted. Fig 2(c) reports the cost of max and swap (row and column exchange) functions, which cause high overheads under all scales taking up $> 30\%$ of the total run-time. These results indicate that matrix inversion is difficult to be efficiently mapped to current high-end CPUs.

Motivated by above insights, we propose a dedicated architecture, the MI2D, that alleviates the bottlenecks of poor data locality and random data manipulations. MI2D features a scalable tile-based Gauss-Jordan Elimination which deploys multi-functional networks to fulfill random data manipulations, so that the expensive row and column exchange operations are hidden. MI2D then further changes the elimination method to transform the computation pattern into standard matrix multiply operations, which greatly improves the

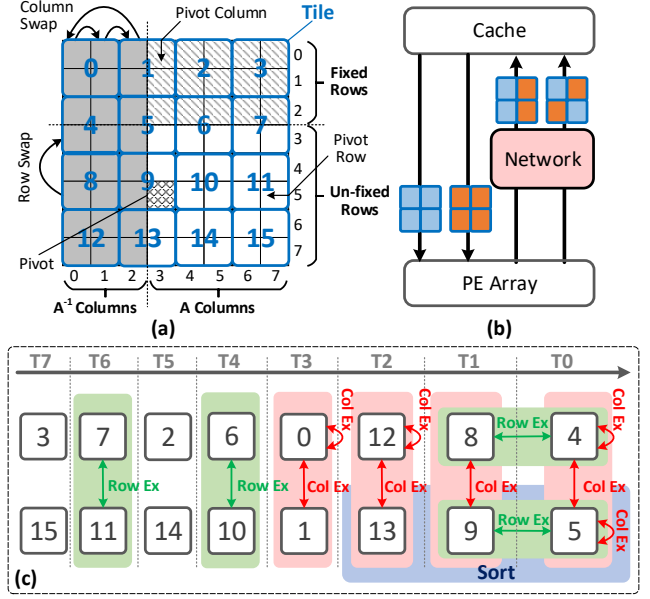


Figure 3: Example of tiled matrix inversion and required column/row operations. MI2D conduct such operations with data reorganization networks that fulfill 2-dimensional cross-tile manipulations when tiles traverse the network.

data reuse. Our evaluations show that MI2D achieves remarkable performance. More importantly, as most recent high-end CPUs and NPUs start to include matrix engines in their designs, our implementation show that MI2D can be easily integrated with existing matrix engines with little overhead, making it a potential solution to provide high-performance matrix inversion capabilities in current matrix processing architectures.

2 2-DIMENSIONAL TILE MANIPULATION

We partition matrix $A \in \mathbb{R}^{N \times N}$ into tiles of $k \times k$ scale. Fig 3(a) gives an example of tile-based matrix inversion. At the given moment, *column*-{0,1,2} of matrix A have been eliminated and replaced by 3 columns of A^{-1} . The ordering of *row*-{0,1,2} is fixed and the other rows are still exchangeable in the future partial pivoting steps. Now, to eliminate *column*-{4}, several kinds of tile manipulations are required: 1) **Column comparison**: all pivot column elements in *tile*-{5,9,13} are compared to select the maximal value (as the next pivot element). 2) **Row exchange**: the selected pivot row elements (*row*-{5}) in *tile*-{8,9,10,11} should be exchanged with *row*-{3} elements in *tile*-{4,5,6,7}. 3) **Column exchange**: the columns of A^{-1} should be reordered to restore the correct sequence. So elements are exchanged between *tile*-{0,4,8,12} and *tile*-{1,5,9,13}. The above operations involve multiple tiles on both dimensions of column and row. To handle such 2-dimensional random data manipulations, MI2D uses a special on-chip network that transfers data while performing transformations, which is inspired by the processing-in-transmission scheme [20]. As depicted in Fig 3(b), the network is inserted between cache (or local memory) and PE array to fulfill the data manipulation across tiles and leaves the PEs to focus on intensive regular computations. Specifically, at each cycle the network takes in multiple tiles and perform data

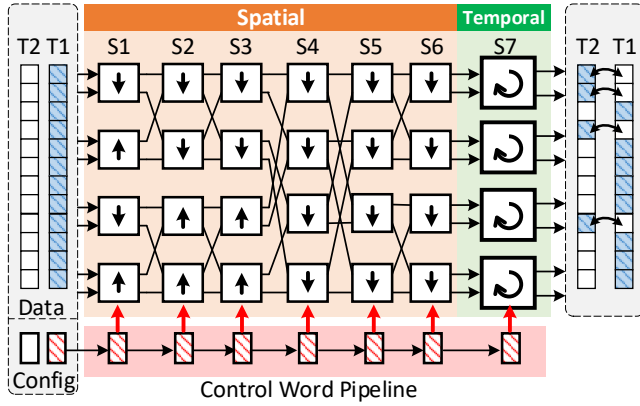


Figure 4: Structure of data reorganization network which includes a Bitonic Sorting Network for spatial manipulation and multiple temporal switch units that perform temporal manipulation. The traversing of data and control words are in pipeline.

manipulations including sorting, random row exchange and column exchange. Moreover, such operations can be done on both spatial adjacent tiles (arrival in the same cycle) and temporal adjacent tiles (arrival in sequential cycles), allowing MI2D to flexibly conduct two-dimensional manipulations as shown in Fig 3(c).

2.1 Network Design

The proposed data reorganization network is a 2-radix on-chip network [8] whose basic blocks are 2-radix switch units. Each unit can route two input signals to two output ports according to routing algorithms. By configuring routing behavior of each unit, the network can perform different transformations while data traverse the network. The 2-radix networks are favorable in hardware design due to their regular topology. As shown in Fig 4, the backbone of our proposed network is pipeline Bitonic Sorting Network, which implements bitonic sorting algorithm to sort elements in parallel [17]. BSN is responsible to conduct spatial reorganization on tiles that simultaneously enter the network. Moreover, in order to manipulate elements that arrive at different cycles, BSN is followed by temporal switch units that can selectively alter the sequence of pipeline stream so that signals of different cycles can be compared and exchanged. The network is controlled by configuration signals (i.e. the control words) which defines the routing behavior of each stage, as depicted in Fig 4. As the network is full pipeline, at each clock cycle it can accept a new group of tiles with the desired transformation command which is described in control words.

2.2 Tile Manipulation Methods

2.2.1 Row/Column Reordering. Consider the tile size is $k \times k$. We assume a $2k^2$ scale network which takes in two tiles at each cycle. Fig 5(a) listed all the required row/column exchange patterns in matrix inversion. For patterns that require one or two tiles, we use the spatial network to perform arbitrary reordering of all the input elements. Specifically, we attach each input element with a tag that indicates its destination as $[tile-index, row-index, col-index]$. Then the spatial network fulfills the desired reorganization by sorting the tags. On the other hand, the pattern gets more complex if cross-tile row exchange and cross-tile column reordering are both required,

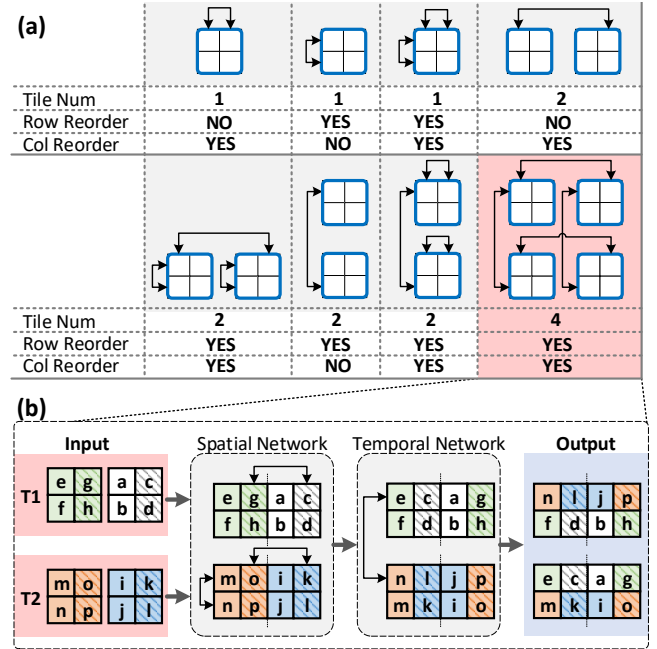


Figure 5: List of row/column reordering patterns in MI2D with detailed explanation of 4-tile manipulation pattern for cross-tile column/row reordering.

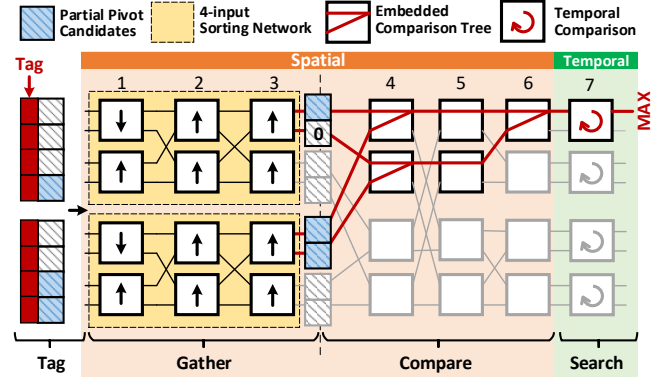


Figure 6: Searching for the pivot cross multiple tiles. The network is extended with a comparator tree embedded in switch units.

where we need to process four tiles on quadrangular positions. This situation occurs when the bottom tiles include the pivot row that should be moved to the top tiles, while left-side tiles should exchange some columns with right-side tiles. As depicted in Fig 5(b), the four tiles are fed to the network in two cycles and undergo a two-step process. At the spatial transform step, all column exchanges are performed. Meanwhile, to swap rows, the network will adjust the position of the bottom row to be aligned with the top row, as shown in Fig 5. Then in the temporal transform step, the correspondent top and bottom rows are exchanged. These steps are fulfilled as non-blocking pipeline, fulfilled within one network traversal.

2.2.2 Maximal Value (Pivot) Search. Another cross-tile operation is searching the maximal value in several tile columns. MI2D fulfills the searching in a gather-compare scheme as depicted in Fig 6. Specifically, the network finds the maximal pivot value from a

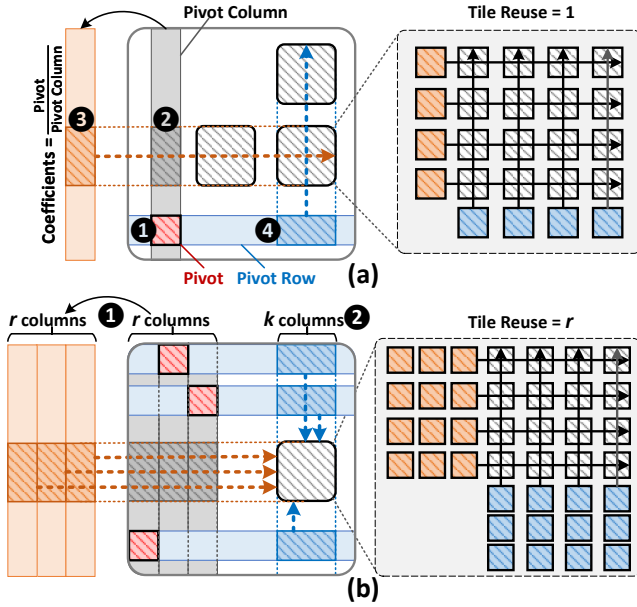


Figure 7: Examples of data dependency and the proposed tile reuse policy by pre-computing multiple columns of coefficients and using them to run multiple iterations on tiles.

series of input tiles with four steps: **Step-1 (Tag)**: Tag the pivot candidates in both tiles with value 1, while the other elements are tagged 0. **Step-2 (Gather)**: The former stages of network are used as two independent k^2 -input BSN that reorder each tile so that the pivot candidates are gathered to the top. **Step-3 (Compare)**: The pivot candidates undergo the embedded comparison tree to find the maximal value. **Step-4 (Search)**: The temporal switch unit compares the selected max value of each cycle, and outputs the maximal value after a period of cycles. After all tiles of pivot column have traversed the network, the pivot value is found.

3 TILE-BASED MATRIX INVERSION

With the proposed network, we remove the burden of irregular tile operations from the computation units. In this section, we further study data dependency of Gauss Elimination and propose an optimized method that greatly improves data reuse.

3.1 Reused Tile Elimination

According to Eq 2-4, Fig 7(a) depicts a tile's computation process and data dependency during one iteration. For each tile with size of $k \times k$, to fulfill the elimination of each iteration in Eq 2-4, it depends on the following data: 1) **pivot element**; 2) k **pivot column elements** that are in the same rows of tile; 3) k **coefficients** calculated by pivot value and k pivot column elements using Eq 2; 4) k **pivot row elements** that are in the same columns of tile. The elimination is conducted by subtracting each tile element with its corresponding pivot row element modified by coefficient, as depicted in Fig 7(a). In this process, each coefficient is reused to compute tiles of the same row, and each pivot row element is reused for a column of tiles. But each tile is used only once.

Meanwhile, we find that each tile only has direct dependency on the corresponding k pivot row elements and k coefficient elements

which are pre-computed using the pivot element and pivot column elements. Therefore, we can pre-compute and store the coefficients of multiple iterations, and use them to repeatedly compute one tile for several iterations. Meanwhile, to guarantee the tile can always get the corresponding pivot row elements for each iteration, we fetch a column of tiles at a time and compute them as a group so that their pivot row elements are always available. Assume we pre-computed and saved coefficients columns for r iterations, then each tile can be reused for r times. After all tiles are computed, total number of r columns of matrix A is eliminated. Fig 7(b) gives an example of the proposed method, which follows the steps:

Step-1: Starting from matrix A 's first column, load r columns $col_0 - col_{r-1}$ to on-chip memory (or the cache), run GJE iteration (GJE-1-GJE-4) for r times and store the generated r columns of coefficients;

Step-2: Load a new column of tiles to on-chip memory and for each tile in the column run elimination (Eq 4 of GJE-4) for r times;

Step-3: Repeat step-2 until all tiles are computed;

Step-4: Repeat step 1-3 starting with next r columns ($col_r - col_{2r-1}$).

We can notice that the repeated elimination depicted in Fig 7(b) includes the interleaved multiply-and-add operations on a $k \times r$ shape coefficient matrix and a $r \times k$ shape matrix of pivot row elements, which resembles the pattern of matrix multiplication. Actually, such computation can be naturally fulfilled by standard matrix multiplication units, which are commonly used in recent high-end CPUs[12] and NPU's, e.g. the matrix engine in Huawei Ascend[10] and systolic-array used in TPU[13]. As to be explained later, our approach can be easily integrated with mainstream matrix engine accelerators with low overheads.

3.2 On-chip Memory Requirement

MI2D can achieve non-blocking computation by prefetching tiles during computation. We must guarantee the reuse factor r is large enough to create long computation time for each tile so that the prefetch overhead can be fully hidden. This also gives requirements on the on-chip memory size. We consider the on-chip memory has effective read bandwidth B_r and write bandwidth B_w , and the throughput of PE arrays is C . For FP32 values, reuse factor r must satisfy the following condition to guarantee full PE usage:

$$\frac{r}{C} \geq \frac{1}{B_w} + \frac{1}{B_r}, \text{ while } r \geq \frac{C(B_w + B_r)}{B_w B_r} \quad (5)$$

The on-chip memory should be able to host r matrix columns as well as r coefficient columns to finish step 1. Considering that the granularity of computation is tile, step 1 requires $\lceil \frac{r}{k} \rceil$ columns of tiles. Meanwhile, to host the prefetched tiles in step 2, there should be additional memory space for at least one column of tiles. To sum up, for a given matrix size $N \times N$ the condition of on-chip memory capacity that guarantees an unblocked computation flow should be:

$$M \geq k \cdot (2\lceil \frac{r}{k} \rceil + 1) \cdot N \cdot 4 = 8kN \lceil \frac{C(B_w + B_r)}{B_w B_r k} \rceil + 4kN \quad (6)$$

where M is the condition of on-chip memory size measured in bytes. As the value of $4kN$ is insignificant compared with the size of M , we can estimate the minimal memory requirement M^* as:

$$M^* \approx 8kN \lceil \frac{C(B_w + B_r)}{B_w B_r k} \rceil \quad (7)$$

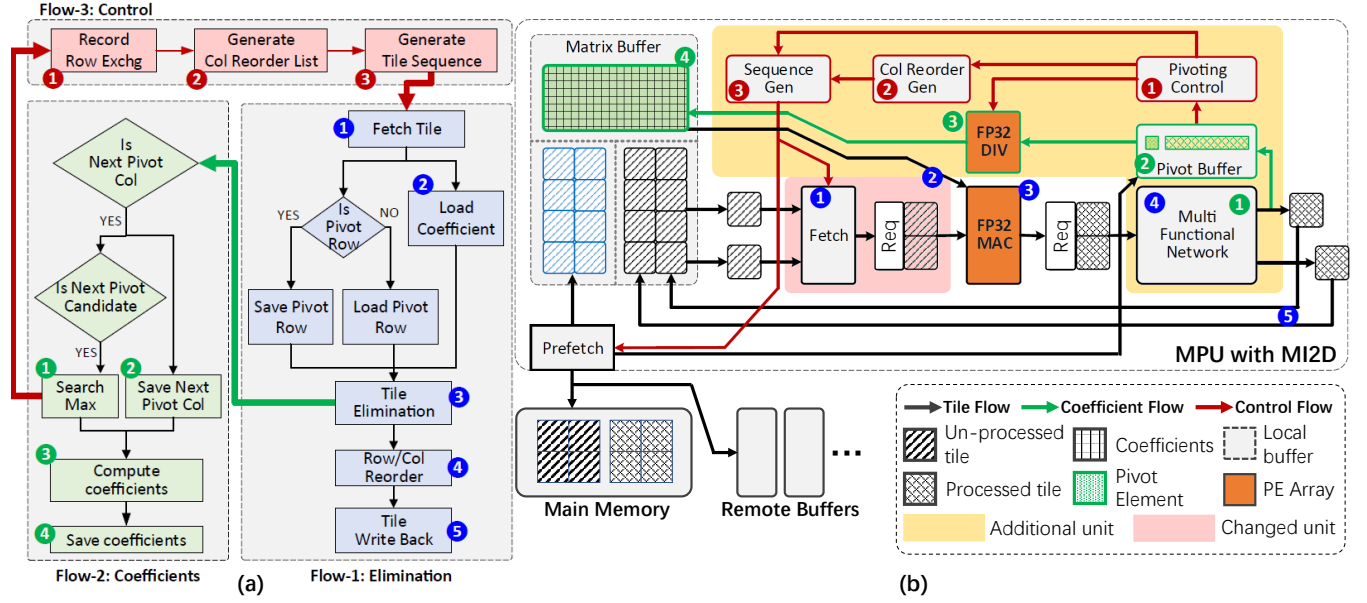


Figure 8: The overall architecture design of MI2D which includes three parallel process flow: the tile computation flow, the coefficient computation flow and the control flow.

For a given matrix size, MI2D requires minimal M^* on-chip memory to reach unblocking computation. Meanwhile, with a determined memory $\hat{M}$, the maximal matrix size for full-speed acceleration is:

$$N^* \approx \frac{\hat{M}}{8k \lceil \frac{C(B_w + B_r)}{B_w B_r k} \rceil} \quad (8)$$

The above equation indicates that MI2D can provide highly-efficient computation on large matrices with relatively small on-chip memory. For example, with tile size $k = 8$, DDR4 bandwidth 40GB/s and 1MB on-chip memory which are moderate hardware settings, a single MI2D core can fully accelerate matrices as large as 12k×12k, while almost all the memory access overheads are hidden.

On the other hand, when a matrix exceeds N^* , it can be partitioned and computed by multiple MI2D cores. Specifically, we horizontally divide the matrix and compute each partition by an independent MI2D core. Since partial pivoting requires operations in column dimension, there are inevitable data communication, which are mainly the pivot value and pivot row elements that are in small size. To minimize the synchronization cost, we pre-compute and send in advance the pivot values of the next iteration so that the inter-core communication is hidden by execution of current elimination. As the inter-communication overhead is generally less than the computation runtime of each elimination iteration, the workflow of MI2D cores is actually rarely stalled.

4 OVERALL ARCHITECTURE

Fig 8 depicts the MI2D architecture. As described in Fig 8(a), MI2D performs matrix inversion with three parallel processing flows. The *elimination flow* is the main computation workload in MI2D. Fig 8(b) depicts the flow in multiple functional modules. This flow starts from the fetch module, which reads two tiles from the matrix buffer at each cycle and tags each tile with its required operation. Guided

by the *Req*, MI2D loads the coefficients, performs the desired elimination and instructs the network to perform tile manipulations. In parallel with the elimination flow, the coefficients of next elimination iteration are pre-computed by the *coefficient flow*. Whenever a tile is eliminated, the next pivot columns are saved and compared through the multi-functional network until the next pivot value is found. Then the elimination coefficients for the next iterations are pre-computed with the floating-point division unit (*DIV*). Meanwhile, the *control flow* is a central controller, which is responsible to determine the processing sequence and desired operations of each tile. This flow first monitors the entire matrix inversion and records the position (row/col ID) of every pivot value, because the position of pivot values determines the rows to exchange and the permuted order of A^{-1} 's columns. Based on this record, the *col_reorder_gen* module calculates the tiles whose columns should be reordered and generate a list of tile sequence. Then the *sequence_gen* considers the necessary tile row-exchange and column-reorder, and uses a dedicated scheduling policy to decide the final sequence for the prefetch module as well as the fetch module.

The scheduling policy determines the sequence of tiles to be fetched, computed and written back. Generally, tiles of the same column should be processed together so that the pivot row elements in this column can be reused and discarded. Assuming the reuse factor is r , we compute the sequence with two priority levels. First, select the column of tiles with the following priority: 1) Columns that contain the next r pivot columns in order to conduct r iterations; 2) Columns of A^{-1} that require inter-tile column reordering. 3) Other columns. Then, for the selected column, pick tiles with following priority: 1) Tiles that contain pivot row elements together with the tiles to swap rows; 2) Tiles that require inter-tile column reordering 3) Tiles that contain pivot candidates for next iteration; 4) Other tiles. In case of multi-core execution, the above steps are

Table 2: Summary of the MI2D implementation cost (area and power) on a standard matrix processing unit (MPU).

	module	area (μm^2)	prop.	power(mW)	prop.
MPU	PE (matrix)	580,064	22%	65.8	30%
	SRAM	1,701,635	64%	86.4	39%
	Logic (mpu)	101,614	4%	17.4	8%
MI2D	PE (div)	6,5099	2%	12.0	5%
	SRAM (pivot)	41,227	2%	5.3	2%
	Logic (MI2D)	100,658	4%	17.0	8%
	Network	69,855	3%	15.9	7%
	Total	2,524,311		219.9	

used to schedule for each MI2D partition with extra checks on the shared pivot rows to guarantee the synchronization.

It should be noted that MI2D is designed to be compatible with matrix processing which are widely provided in NPUs. Fig 8 shows the modules that need to be added or modified on a matrix processing unit. The data fetch unit and changed to received commands from the *control flow* and to attach *Req* tags to tiles. And several extra units are added to provide data reorganization, pre-computation of coefficients and scheduling policy. However, the most central and resource-costly parts, i.e., the matrix multiply-and-add units and the on-chip buffer, are directly reused without any change. The accelerator can choose to enable MI2D to perform efficient matrix inversions, and otherwise remains to be a standard matrix processing unit. As to be shown in next section, MI2D components only impose light overheads on the overall architecture.

5 EVALUATIONS

5.1 Implementation

We implement MI2D on a standard matrix processing unit (MPU) used in an open-source ASIC NPU core [18], which applies an $2 \times 8 \times 8$ scale systolic array to perform matrix multiplications. Besides MPU, the core also includes 1MB size of on-chip buffer and a simple RISC-V CPU to issue commands. Based on this core, we add 8-width division units, a 4KB pivot buffer, an 128-input scale network and several control blocks shown in Fig 8. The tile size of MI2D is 8×8 . We set the core's working frequency to be 800 MHz and synthesize the core using Synopsys Design Compiler (DC) with TSMC 28nm High-Performance Standard Cell Library. The SRAM blocks are also generated with the same library. The computation units of single-precision MAC, division and comparator are referring to the Synopsys DesignWare foundation IP libraries. The synthesize results are listed in Table 2, which indicates that MI2D components only add to 10% of the total area. Meanwhile, though the power overhead is a little higher (23%), most of the cost is dynamic power consumption due to the high-throughput on-chip network transmission. Therefore, we can apply the clock-gating technique to turn off the MI2D circuits when matrix inversion is not needed, so that most of the extra power cost can be avoided when the MPU handles traditional computation tasks. Hence, by converting matrix inversion into matrix multiplications, MI2D can be easily integrated into the matrix processing unit by efficiently reusing its storage and computation resources with little extra overheads, which means MI2D can be widely used in MPUs to provide high-performance matrix inversion capability.

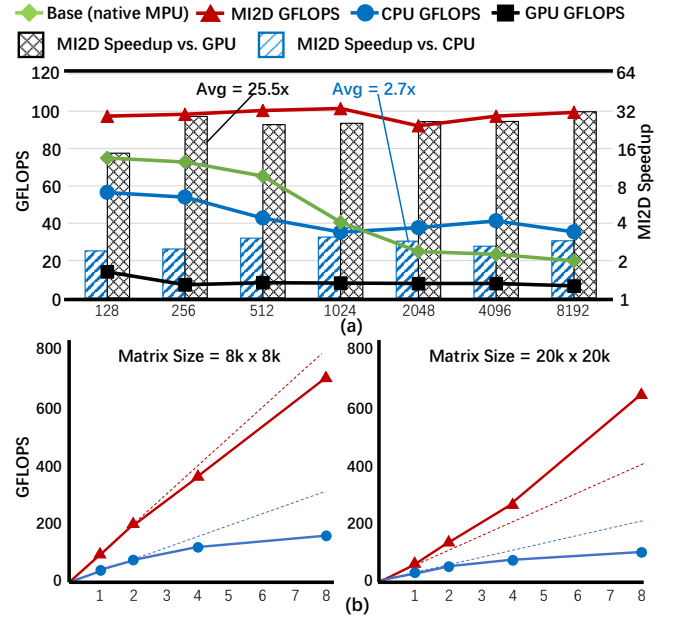


Figure 9: Comparison of MI2D and other methods on various matrix scales for: (a) single-thread; (b) multi-thread using 1/2/4/8 cores.

5.2 Performance Evaluation

We test the performance of MI2D using cycle-accurate circuit simulation with DDR4 model. We also test the performance of native MPU core without MI2D as the baseline. On the other hand, most existing research works on ASIC or FPGA [7][6][4][9] are either not open-sourced, or lacks comparable performance results in their papers, which makes it impossible to perform convincing comparisons. Therefore, we decide to compare our design with the matrix inversion implementations of main-stream high-performance platforms CPU/GPU using their state-of-the-art linear algebra libraries, i.e. Intel MKL, and cuBLAS. Specifically, we evaluate MKL (version 2019.3) on Intel XEON Gold 6146 CPU which is working at 4.2GHz with 24.75MB L3 Cache and DDR4. The NVIDIA cuBLAS library (version 10.1) is tested with CUDA (version 10.1) on the NVIDIA RTX 2080 Ti platform, which works at 1.4GHz frequency with GDDR6. The benchmark is a series of random single floating-point dense square matrices scaling from 128 to 8192.

Fig 9(a) lists the results of single-core execution. As depicted, the native MPU's performance drops fast on larger matrices as its simple pipeline is inadequate to process random large-scale data reorganizations. The cuBLAS kernel has lowest performance as GPU is not optimized to handle irregular data operations. MKL shows better performance than both native MPU and cuBLAS on large matrices as it has several hardware optimizations to alleviate the memory bound, such as the hardware prefetcher and out-of-order execution. However, its performance still declines on larger scales. Meanwhile, MI2D can greatly improves MPU's performance and achieves superior performance over MKL and cuBLAS on all matrix scales. Generally, MI2D achieves a steady and average $2.7 \times$ speedup against MKL and average $25.5 \times$ speedup against cuBLAS.

Moreover, we test the multi-thread performance on large matrices. The inter-core bandwidth among MI2D cores is set to be

2GB/s. We choose two matrix scales: 8k that can be fully accelerated by one MI2D, and 20k that has 1.6 GB data and far exceeds the on-chip memory capacity of both MI2D and CPU. We experiment with 1/2/4/8 threads and compares their performance. As depicted in Fig 9(b), MKL is not scaling well on both matrices as its performance fails to achieve linear growth with the number of cores. On the other hand, MI2D presents a near-linear performance gain on the 8k matrix and a superlinear curve on the 20k matrix. This is because 8k matrix can be fully accelerated by single MI2D core so that the inter-thread communication would slightly undermine the usage rate of MI2D cores. On the other hand, 20k matrix exceeds the capability of one MI2D core and achieves better efficiency when computed by multiple MI2D cores. The results indicate MI2D has a good scalability that allows it to process very large matrices with high parallel efficiency.

6 CONCLUSION

In this paper, we propose MI2D, a novel matrix inversion accelerator. By using 2-dimensional cross-tile operations supported by a dedicated multi-functional on-chip network and a novel algorithm for efficient data reuse, MI2D transforms the matrix inversion into regular matrix multiplication patterns, meaning that it can be efficiently integrated with matrix processing units in modern CPU and NPUs with limited resource overheads. Experiments show that MI2D achieves high performance matrix inversion with 2.7× speedup against the Intel Skylake CPU, and 24× against the NVIDIA RTX 2080 Ti.

ACKNOWLEDGMENTS

This work was supported in part by the National Key Research and Development Program of China (Program No. 2021YFB0300900), and the Key Research and Development Program of Shaanxi (Program No. 2022ZDLGY01-08).

REFERENCES

- [1] A C Antoulas and Danny C Sorensen. 2001. Approximation of large-scale dynamical systems: an overview. *International Journal of Applied Mathematics and Computer Science* 11, 5 (2001), 1093–1121.
- [2] Janier Arias-García, Ricardo Pezzuol Jacobi, Llanos, and Carlos H. 2011. A suitable FPGA implementation of floating-point matrix inversion based on Gauss-Jordan elimination. In *VII Southern Conference on Programmable Logic*. IEEE, 263–268.
- [3] Peter Benner, Pablo Ezzatti, Enrique S Quintana-Ortí, and Alfredo Remón. 2013. Matrix inversion on CPU–GPU platforms with applications in control theory. *Concurrency and Computation: Practice and Experience* 25, 8 (2013), 1170–1182.
- [4] David Boland. 2016. Reducing memory requirements for high-performance and numerically stable gaussian elimination. In *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. ACM, 244–253.
- [5] Debabrata DasGupta et al. 2013. In-place matrix inversion by modified Gauss-Jordan algorithm. *Applied Mathematics* 4, 10 (2013), 1392–1396.
- [6] Goncalo M de Matos and Horacio C Neto. 2007. Memory optimized architecture for efficient gauss-jordan matrix inversion. In *2007 3rd Southern Conference on Programmable Logic*. IEEE, 33–38.
- [7] Rui Duarte, Horácio Neto, and Mário Véstias. 2009. Double-precision gauss-jordan algorithm with partial pivoting on fpgas. In *2009 12th Euromicro Conference on Digital System Design, Architectures, Methods and Tools*. IEEE, 273–280.
- [8] Natalie Enright Jerger, Tushar Krishna, and Li-Shiuan Peh. 2017. On-chip networks. *Synthesis Lectures on Computer Architecture* 12, 3 (2017), 1–210.
- [9] Zhenhua Jiang and Sayed Ata Raziei. 2017. An efficient FPGA-based direct linear solver. In *2017 IEEE National Aerospace and Electronics Conference (NAECON)*. IEEE, 159–166.
- [10] Heng Liao, Jiajin Tu, Jing Xia, Hu Liu, Xiping Zhou, Honghui Yuan, and Yuxing Hu. 2021. Ascend: a scalable and unified architecture for ubiquitous deep neural network computing: Industry track paper. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 789–801.
- [11] Ming Liu, Matthieu Crussiere, and Jeanfrancois Helard. 2012. A Novel Data-Aided Channel Estimation With Reduced Complexity for TDS-OFDM Systems. *IEEE Transactions on Broadcasting* 58, 2 (2012), 247–260.
- [12] Nevine Nassif, Ashley O Munch, Carleton L Molnar, Gerald Pasdast, Sitaraman V Lyer, Zibing Yang, Oscar Mendoza, Mark Huddart, Srikrishnan Venkataraman, Sireesha Kandula, et al. 2022. Sapphire Rapids: The Next-Generation Intel Xeon Scalable Processor. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. IEEE, 44–46.
- [13] Thomas Norrie, Nishant Patil, Doe Hyun Yoon, George Kurian, Sheng Li, James Laudon, Cliff Young, Norman P Jouppi, and David A Patterson. 2020. Google's Training Chips Revealed: TPuv2 and TPuv3.. In *Hot Chips Symposium*. 1–70.
- [14] Kwanjun Park, Taeseok Daniel Yang, Hyung Jin Kim, Taedong Kong, Jung Min Lee, Hyuk Soon Choi, Hoon Jai Chun, Beopmin Kim, and Youngwoon Choi. 2019. Inversion-free image recovery from strong aberration using a minimally sampled transmission matrix. *Scientific Reports* 9, 1 (2019), 1206.
- [15] James Reinders. 2012. An overview of programming for Intel Xeon processors and Intel Xeon Phi coprocessors. *Intel Corporation, Santa Clara* 1 (2012), 1550002.
- [16] Qing Wang, Ming Liu, Nian Liu, and Zhangdui Zhong. 2018. On Augmenting UL Connections in Massive MIMO System Using Composite Channel Estimation. (2018), 1–7.
- [17] Tian Xia, Pengchen Zong, Haoran Zhao, Jianming Tong, Wenzhe Zhao, Nanning Zheng, and Pengju Ren. 2020. COCOA: Content-Oriented Configurable Architecture Based on Highly-Adaptive Data Transmission Networks. In *Proceedings of the 2020 on Great Lakes Symposium on VLSI*. 253–258.
- [18] Wenzhe Zhao. 2021. Open-HiPU200 Project on github.com/xjtuiar-cag/Open-HiPU200. https://www.researchgate.net/publication/243771074_DIY_Corpora_the_WWW_and_the_Translator
- [19] Dengkui Zhu, Boyu Li, and Ping Liang. 2015. On the matrix inversion approximation based on neumann series in massive MIMO systems. (2015), 1763–1769.
- [20] Pengchen Zong, Tian Xia, Haoran Zhao, Jianming Tong, Wenzhe Zhao, Nanning Zheng, and Pengju Ren. 2020. PIT: Processing-In-Transmission with Fine-Grained Data Manipulation Networks. *IEEE Trans. Comput.* (2020).